

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 10%

QUESTIONS: 10

Annexure A

Total Marks:20

MOCK INTERVIEW QUESTIONS

Code of Conduct:

*I Moshitha Kolanjiyappan (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Moshitha k

Annexure A

Short Answer Test

1. You are working as a data analyst in a hospital where patient data is continuously collected and stored in CSV files. However, the dataset contains missing values, inconsistent units, and duplicate entries. The system must process this data in near real-time for reporting. Explain how you would design a solution using NumPy and Pandas to clean, filter, and standardize the data efficiently. Discuss indexing, aggregation, and saving the processed data, along with strategies to handle unexpected data anomalies.

Answer:

A near real-time hospital data-processing system can be designed using Pandas for DataFrame operations and NumPy for efficient numerical calculations. The CSV files can be read using `pd.read_csv()`. Since new data is continuously generated, the system can process files in small batches instead of loading the complete historical dataset into memory.

The first step is to inspect the data using `head()`, `info()`, `shape`, and `isnull().sum()`. Missing numerical values such as temperature, blood pressure, or glucose can be filled using suitable statistics such as the median. Missing categorical values can be replaced with "Unknown" when appropriate.

Duplicate records can be detected using `duplicated()` and removed using `drop_duplicates()`. Inconsistent units should be converted into a common unit before analysis. For example, temperature values should be converted to the same unit and weight measurements should use one standard unit.

Filtering can be used to remove medically impossible or invalid values. Boolean indexing can identify records outside predefined valid ranges. The patient ID or timestamp can be used as an index when efficient record lookup is required.

Aggregation can calculate statistics such as average patient temperature, minimum and maximum blood pressure, and the number of patients in different categories. Grouping by department, age group, or diagnosis can produce useful reports.

After cleaning, the processed DataFrame can be saved using `to_csv()`. Unexpected anomalies should be logged rather than silently deleted. The system should maintain an error log containing the patient record, affected column, invalid value, and reason for rejection.

NumPy vectorized operations should be used for numerical calculations because they are faster than Python loops. Processing data in batches also reduces memory usage and supports near real-time reporting.

2. A financial company provides you with millions of transaction records that must be analyzed to detect unusual spending patterns. Due to system limitations, memory and computation time are critical constraints. Describe how you would use NumPy arrays and Pandas DataFrames to perform filtering, grouping, and statistical analysis efficiently. Explain how you would optimize indexing and aggregation operations, and how you would debug performance bottlenecks or incorrect outputs.

Answer:

For millions of financial transactions, memory-efficient processing is essential. Pandas DataFrames can be used for structured transaction data, while NumPy arrays can perform fast numerical calculations.

The data should be loaded using only the required columns. For very large CSV files, Pandas `read_csv()` can use `usecols` and `chunksize` to process the file in smaller batches. Appropriate data types such as `float32`, `int32`, or categorical types can reduce memory usage.

Filtering can identify unusual transactions based on amount, location, transaction frequency, or time. Boolean indexing should be used because it is faster than processing records individually.

Transactions can be grouped by customer ID, account ID, or merchant. Aggregation functions such as `sum()`, `mean()`, `count()`, `min()`, and `max()` can identify spending patterns. For example, unusually high total spending or a large number of transactions within a short period can be flagged.

Indexes should be created only when they provide a performance benefit, such as frequent searches by customer ID or timestamp. Unnecessary indexes can increase memory consumption.

NumPy can calculate statistical measures such as mean, standard deviation, and percentiles efficiently. Z-scores or IQR can be used to detect unusual transaction amounts.

Performance problems can be debugged by measuring execution time, checking memory usage, inspecting DataFrame dimensions, and identifying unnecessary copies or loops. Incorrect outputs can be investigated by checking filtering conditions, group sizes, missing values, and sample records manually.

3. You are given multiple datasets from different departments (sales, marketing, and customer support), each stored in different file formats. Your task is to combine these datasets into a unified DataFrame while ensuring no loss of critical information and maintaining consistent indexing. Explain your approach using Pandas for file I/O, combining, grouping, and sorting. How would you validate the merged dataset and resolve conflicts such as missing or duplicate keys?

Answer:

The first step is to identify the formats of the datasets. CSV files can be loaded using `pd.read_csv()`, Excel files using `pd.read_excel()`, and JSON files using `pd.read_json()` or `pd.json_normalize()` for nested data.

After loading the files, column names and data types should be standardized. The common key, such as Customer ID, should have the same data type and format in every DataFrame.

Before merging, duplicate keys should be detected using `duplicated()` and `groupby()`. Missing keys should also be identified. Records with missing keys should be investigated because the key is required for accurate merging.

The datasets can then be combined using Pandas `merge()`. If preserving all valid information is important, an outer merge can be used initially. The `indicator=True` option can help identify records that exist only in one source.

Conflicting values should be resolved using predefined rules. For example, the most recent record may be preferred, or one department may be considered the authoritative source for a particular attribute.

After merging, the data should be grouped by the common key to detect unexpected duplicates. Sorting by the key provides consistent ordering, and `reset_index()` creates a clean index.

Validation should compare the number of records before and after merging, check missing keys, identify duplicate keys, and verify that all required columns are present.

4. A company requires an automated system that reads daily data files, processes them, and generates summary reports along with visualizations. The pipeline must filter relevant data, compute statistics, and produce sorted outputs for dashboards. Describe how you would design this workflow using NumPy and Pandas, including file handling, filtering, aggregation, and plotting. Also explain how you would handle errors such as corrupted files, inconsistent formats, or missing columns.

Answer:

An automated daily data-analysis pipeline should contain several stages: file detection, loading, validation, cleaning, filtering, aggregation, visualization, and report generation.

The system can identify newly available files and load them using Pandas. Different formats can be handled using appropriate functions such as `read_csv()`, `read_excel()`, or `read_json()`.

After loading, the pipeline should verify that all required columns are present. Data types, missing values, duplicates, and invalid records should also be checked.

Filtering can select only relevant records using Boolean indexing. NumPy can be used for efficient numerical calculations, while Pandas can perform grouping and aggregation.

For example, data can be grouped by product category and summarized using total, average, minimum, and maximum values. The resulting DataFrame can be sorted according to the metric required by the dashboard.

Visualization can be performed using Matplotlib. Bar charts can show category comparisons, line charts can show trends, and histograms can show distributions.

File errors should be handled using exception handling. Corrupted files should be moved to an error location or recorded in an error log. Inconsistent formats should be standardized during preprocessing. Missing required columns should generate a validation error rather than allowing the pipeline to produce an incorrect report.

The final summary tables and plots can be saved automatically for dashboard use.

5. You are working on an e-commerce platform where customer behavior data is collected in real time. The dataset contains clickstream data with missing values and inconsistent formats. Explain how you would preprocess and structure this data using Pandas. Discuss filtering, feature extraction, indexing strategies, and how you would prepare the data for downstream machine learning tasks.

Answer:

Clickstream data records user activities such as page views, product clicks, searches, cart additions, and purchases. Pandas can be used to organize and clean this information.

The first step is to load the data and standardize column names and data types. Timestamp columns should be converted using `pd.to_datetime()`. Missing values should be handled according to the meaning of each column.

Duplicate events can be removed using event IDs or suitable combinations of user ID, timestamp, and event type. Invalid records can be filtered using Boolean conditions.

Feature extraction is an important step. New features such as session duration, number of clicks, number of products viewed, cart additions, and purchase frequency can be calculated using grouping and aggregation.

The timestamp can be used as an index for time-based analysis. User ID can be used for grouping users and calculating behavioral statistics.

Categorical variables such as device type and event type can be encoded for machine learning. Numerical features can be normalized using NumPy or Pandas operations.

Finally, the processed dataset should be checked for missing values, duplicate records, incorrect types, and invalid values before being passed to a machine-learning model.

6. A healthcare analytics system requires continuous monitoring of patient vitals stored as time-series data. The dataset includes irregular timestamps and noisy readings. Describe how you would use NumPy and Pandas to clean, resample, and analyze this data. Explain how rolling statistics and aggregation functions can help identify abnormal patterns.

Answer:

Patient vital data such as heart rate, blood pressure, oxygen saturation, and temperature can be processed using Pandas time-series functionality and NumPy numerical operations.

First, the timestamp column should be converted into a datetime format using `pd.to_datetime()`. Invalid timestamps should be identified and removed or corrected. The records should then be sorted chronologically and the timestamp should be used as the DataFrame index.

Irregular observations can be converted to regular intervals using Pandas `resample()`. Depending on the measurement, aggregation functions such as mean, minimum, maximum, or median can be used.

Noisy readings can be detected using statistical techniques such as Z-score or IQR. Clearly invalid measurements should be filtered, while genuine extreme observations should be investigated rather than automatically deleted.

Rolling statistics can identify changes over time. A rolling mean can show the patient's recent average vital value, while rolling standard deviation can identify increased variability.

For example, a sudden increase in rolling heart rate or a sustained decrease in oxygen saturation can be flagged for further investigation.

NumPy provides efficient numerical calculations, while Pandas manages timestamps, indexing, grouping, and rolling windows.

7. You are given a dataset with millions of rows containing mixed data types and categorical variables. The goal is to prepare this data for training a machine learning model under strict time constraints. Explain how you would perform encoding, normalization, and efficient data transformation using Pandas and NumPy. Discuss optimization and debugging strategies.

Answer:

The first step is to inspect the dataset and identify numerical, categorical, date, and text columns. Pandas can be used to inspect data types and missing values.

Numerical missing values can be filled using appropriate statistics such as median, while categorical missing values can be replaced with the mode or an "Unknown" category.

Categorical variables must be converted into numerical representations. One-hot encoding can be used for categories with a small number of values. Pandas categorical types or numerical encoding can reduce memory usage when appropriate.

Numerical features should be normalized or standardized. Min-Max scaling converts values approximately into the range 0 to 1, while Z-score normalization produces values centered around zero.

NumPy vectorized operations should be preferred over Python loops. Unnecessary columns should be removed, and efficient data types such as float32 and int32 can reduce memory usage.

For millions of rows, processing can be performed in chunks when necessary. Debugging should include checking memory usage, execution time, missing values, data types, and the number of records before and after transformations.

8. A logistics company wants to analyze delivery performance using large datasets containing location, time, and traffic data. Explain how you would design a data processing pipeline using Pandas for grouping, filtering, and aggregation. Discuss how indexing and sorting improve query efficiency and how you would validate the accuracy of computed metrics.

Answer:

A logistics data-processing pipeline should begin by loading delivery, location, time, and traffic datasets into Pandas DataFrames. Timestamp columns should be converted to datetime format, and location information should be standardized.

The datasets can be merged using a common delivery or shipment ID. Missing and duplicate IDs should be checked before merging.

Filtering can select deliveries based on region, delivery status, traffic condition, or time period. Grouping can be performed by region, driver, vehicle, or delivery type.

Important metrics can include average delivery time, total deliveries, number of delayed deliveries, and delay percentage. Aggregation functions such as mean(), count(), and sum() can calculate these metrics.

Sorting the data by timestamp or delivery ID creates a consistent order. Indexing frequently searched columns such as delivery ID or timestamp can improve lookup performance.

Accuracy can be validated by comparing aggregated totals with the original records. Sample calculations should be performed manually and compared with program output. Missing records, duplicate deliveries, and incorrect timestamps should also be investigated.

9. You are tasked with building a data validation system for incoming datasets in a machine learning pipeline. The system must check for missing values, invalid ranges, and inconsistent formats. Describe how you would implement validation rules using Pandas, and explain how indexing, filtering, and aggregation help enforce data quality constraints.

Answer:

A data validation system should verify that incoming datasets satisfy predefined quality rules before entering the machine-learning pipeline.

First, Pandas can check whether all required columns exist and whether their data types match the expected schema. Missing values can be identified using `isnull().sum()`.

Numerical columns should be checked against valid minimum and maximum ranges. Boolean filtering can identify records that violate these constraints. Categorical fields should be compared against a list of permitted values.

Dates should be converted into datetime format and invalid dates should be detected. Duplicate records can be identified using `duplicated()`.

Indexing can help uniquely identify records and quickly locate invalid rows. Filtering can separate valid and invalid records. Aggregation can summarize validation results, such as the number of missing values or invalid records per column.

Invalid records should be placed in an error report containing the row identifier, column, invalid value, and reason for failure.

After corrections, the validation rules should be executed again. Only datasets that satisfy all critical constraints should be passed to the machine-learning stage.

10. A company wants to analyze user activity logs to generate insights such as peak usage times and user segmentation. The dataset is large and continuously updated. Explain how you would use NumPy and Pandas to process, filter, and aggregate this data efficiently. Discuss strategies for handling scalability and ensuring real-time performance.

Answer:

User activity logs can contain millions of records generated continuously. Pandas can organize these logs into DataFrames, while NumPy can perform efficient numerical operations.

Timestamps should be converted to datetime format and used for time-based analysis. Additional features such as hour, day, weekday, and month can be extracted from timestamps.

Filtering can select specific users, activities, time periods, or geographic areas. Grouping by hour can identify peak usage times. Grouping by user ID can calculate metrics such as number of sessions, page views, clicks, and purchases.

User segmentation can be based on activity levels. For example, users can be classified as low, medium, or highly active according to the number of events.

For scalability, data should be processed in batches or chunks rather than loading the entire historical dataset into memory. Only required columns should be loaded. Efficient data types and vectorized operations should be used.

Indexes should be created when frequent queries require them. Recent data can be processed separately while older data is maintained as summarized information.

Real-time performance can be monitored by measuring processing time and memory usage. Slow operations, unnecessary copies, and Python loops should be identified and optimized.

11. You are given a small dataset containing student records with missing marks and inconsistent grade formats. How would you use NumPy and Pandas to clean the data, fill missing values, and standardize grades? Explain your approach in simple steps.

Answer:

The student dataset can first be loaded into a Pandas DataFrame. The columns and data types should be checked using `head()`, `info()`, and `isnull().sum()`.

Missing marks can be identified using Pandas missing-value functions. If only a few marks are missing, the missing values can be replaced with the median or mean of the relevant subject.

Grade values should be standardized. For example, "A", "a", and " A " can all be converted to "A" using string cleaning operations.

Duplicate student records should also be checked using `duplicated()`.

NumPy can be used to calculate average marks, maximum marks, minimum marks, and other numerical statistics.

The cleaned DataFrame should be validated to ensure that marks are within valid ranges and grades belong to the approved categories.

Finally, the cleaned dataset can be saved using `to_csv()`.

Simple steps:

- Load student dataset
- Check missing values
- Fill missing marks
- Remove duplicate records
- Clean grade strings
- Convert grades to standard format
- Check valid mark ranges

Calculate required statistics
Reset index
Save cleaned dataset

12. A company provides you with sales data, and you are asked to find the total and average sales for each product category. How would you use Pandas grouping and aggregation functions to solve this problem? Explain the steps clearly.

Answer:

The sales dataset can be loaded into a Pandas DataFrame using `pd.read_csv()`.

The important columns are the product category and sales amount. The sales amount should be converted to a numerical data type if necessary.

The `groupby()` function can then group records according to product category. The `sum()` function calculates total sales, while `mean()` calculates average sales.

For example, the operation can conceptually be written as:

Group sales data by Product_Category
Calculate SUM of Sales
Calculate MEAN of Sales
Display summarized results

The resulting DataFrame contains one row for each product category and columns showing total and average sales.

The results can then be sorted by total sales to identify the best-performing categories. Missing or invalid sales values should be handled before aggregation.

13. You receive a dataset with duplicate customer entries and incorrect data types (e.g., numbers stored as strings). How would you identify and fix these issues using Pandas? Describe the methods you would use for cleaning and validation.

Answer:

The dataset should first be loaded into a Pandas DataFrame. Its structure can be inspected using `info()` and `dtypes`.

Duplicate customer records can be identified using `duplicated()`. The duplicate records should be examined before removal to ensure that they are genuine duplicates. Valid records should not be deleted accidentally.

Incorrect numerical data types can be converted using `pd.to_numeric()`. If invalid strings are found, they can be converted to missing values using appropriate error handling and then corrected or imputed.

Date columns can be converted using `pd.to_datetime()`.

After cleaning, the dataset should be checked again for duplicate records, missing values, and incorrect data types.

Cleaning process:

- Load dataset
- Inspect data types
- Identify duplicate customers
- Review duplicate records
- Remove genuine duplicates
- Convert numerical strings to numbers
- Convert date strings to datetime
- Handle invalid conversion results
- Validate data types
- Check missing values
- Reset index
- Save cleaned dataset

This process ensures that the customer dataset is consistent and suitable for further analysis.

14. A dataset contains timestamps and corresponding values, but the timestamps are not sorted. How would you use Pandas to sort the data and compute a simple moving average? Explain your approach.

Answer:

The timestamp column should first be converted into Pandas datetime format using `pd.to_datetime()`.

The DataFrame should then be sorted according to the timestamp column using `sort_values()`. This is important because a moving average must use observations in chronological order.

The timestamp can optionally be set as the DataFrame index. A rolling window can then be applied to the numerical value column.

For example, for a three-observation moving average:

- Convert timestamp to datetime
- Sort records by timestamp
- Set timestamp as index
- Select value column

Apply rolling window of size 3
Calculate mean
Store result in Moving_Average column

Conceptually:

```
Moving_Average = Value.rolling(window=3).mean()
```

The first two records may contain missing moving-average values because a complete three-observation window is not available.

The final DataFrame can be sorted chronologically and saved for further analysis.

15. You are given a dataset with both numerical and categorical features, and you need to prepare it for a machine learning model. How would you perform basic preprocessing such as handling missing values, encoding categorical variables, and normalizing numerical features using NumPy and Pandas? Explain the steps briefly.

Answer:

The first step is to load the dataset into a Pandas DataFrame and identify numerical and categorical columns.

Missing numerical values can be replaced with the mean or median of the corresponding feature. Missing categorical values can be replaced with the mode or an "Unknown" category.

Categorical variables must be converted into numerical form before being given to most machine-learning algorithms. One-hot encoding can be used for nominal categories. For example, a Gender column containing "Male" and "Female" can be converted into separate binary columns.

Numerical features should then be normalized or standardized. Min-Max scaling can be used to convert values into approximately the range 0 to 1:

$$X' = (X - X_{\min}) / (X_{\max} - X_{\min})$$

Alternatively, Z-score normalization can be used:

$$X' = (X - \text{Mean}) / \text{Standard Deviation}$$

NumPy can perform these numerical transformations efficiently using vectorized operations.

The final dataset should be checked for remaining missing values, incorrect data types, infinite values, and unexpected categories.

Basic preprocessing steps:

Load dataset using Pandas
 Identify numerical and categorical columns
 Check missing values
 Fill numerical missing values
 Fill categorical missing values
 Encode categorical variables
 Normalize numerical features
 Check for invalid values
 Check final DataFrame shape
 Reset index if necessary
 Prepare data for machine learning

For a machine-learning workflow, normalization and encoding parameters should be learned from the training data and then applied consistently to validation and test data to prevent data leakage.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	25	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand